MediChain technical architecture

How the system is built, deployed, secured and verified. Version as deployed on 13-247-143-188.sslip.io, October 2026.

Item	Value
Server	Rust 2021, Actix-web 4, Tokio; about 158,000 lines including tests
Clients	React 18 + TypeScript + Vite + Tailwind + Zustand: a staff portal, a patient app and a shared library (about 119,000 lines)
Database	PostgreSQL 16, 165 tables from one schema baseline; sqlx with bound parameters only
Interface	500+ REST endpoints under /api, plus server-sent events at /api/events
Cryptography crate	ChaCha20-Poly1305, Argon2id, Ed25519, zeroisation of key material
Hosting	AWS af-south-1, defined in CloudFormation (40 resources) and deployed by one script

1. System context

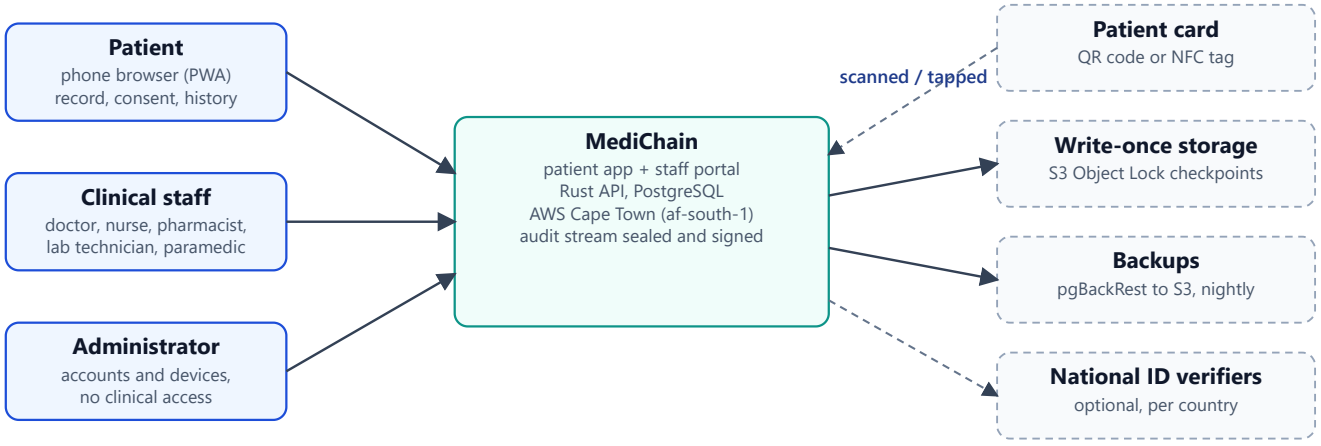


Figure 1. System context: who uses MediChain and what it depends on.

There are two installable web apps (PWAs). The **patient app** is phone-first: record, emergency card, Medical ID QR, access control, consent, medicines, messages. The **staff portal** serves six roles from one codebase; the navigation and every API call are filtered by role. Both talk to the same API and receive live updates over server-sent events.

2. Deployment

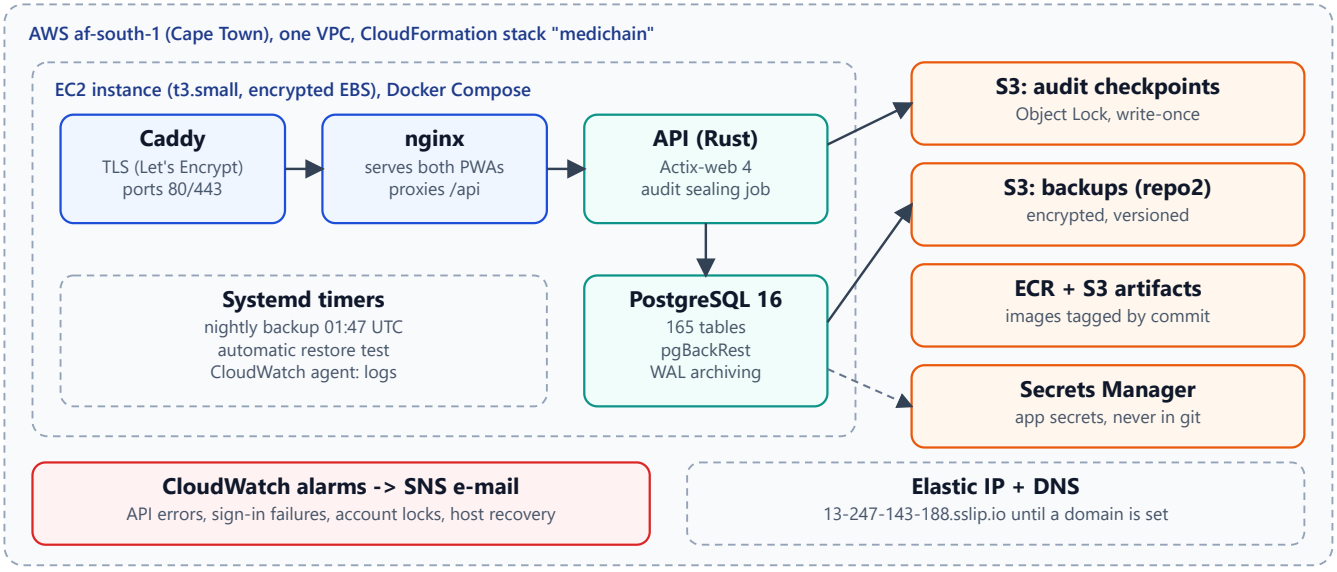


Figure 2. Deployment: everything runs in South Africa; data at rest is encrypted.

A deploy runs five steps: (1) create or update the CloudFormation stack, (2) add any missing application secret on the instance without rotating existing ones, (3) build the API, web and PostgreSQL images and push them to ECR tagged with the git commit, (4) upload the compose bundle to S3, (5) start it on the instance through SSM Run Command. The only inbound ports are 80 and 443; there is no SSH. The instance image is pinned, so a redeploy never replaces the server or its disk.

3. Request lifecycle



Figure 3. Every request passes the same nine checks before its handler runs; the handler then applies role and care-relationship rules.

Inside the handler, the caller comes only from the verified JWT and its durable session; a caller-supplied identity header carries no authority. The handler then checks the role and the relationship to the patient. A handler takes a *request* type, never a domain type, so a form that collects part of a record can still save.

4. Identity and sign-in

1. The browser derives a proof from the password: PBKDF2-SHA512, 210,000 rounds, salted with the login ID, then HKDF-SHA256. The password itself is never sent.
2. The server compares the proof with an Argon2id verifier and issues a JWT bound to a durable, revocable session row.
3. Privileged actions require a fresh authenticator-app (TOTP) code; repeated failures lock the account and raise an alarm.

5. Authorisation and emergency access

Roles rank as Admin, then Doctor and Nurse, then Lab Technician and Pharmacist, then Patient, but a role is necessary rather than sufficient. A record opens only through one of four routes: the patient themself, a patient-approved grant (at most 30 days), a care relationship (appointment, consultation or referral), or an emergency grant.

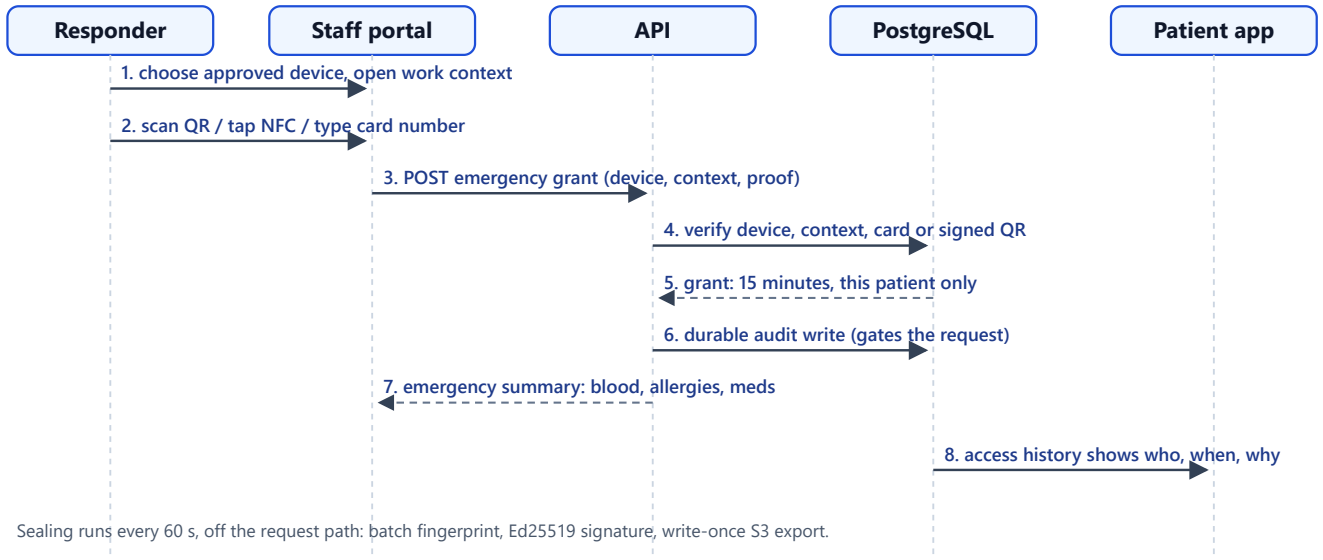


Figure 4. Emergency (break-glass) access. Only step 6, the durable audit row, may refuse a read; sealing never does.

6. The sealed audit stream

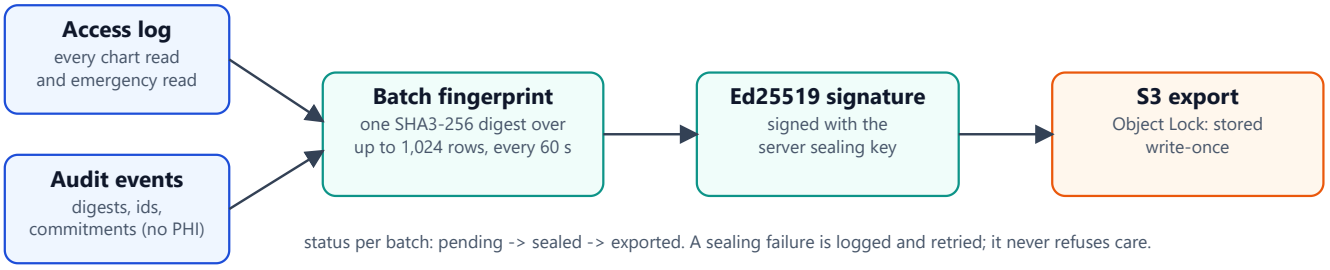


Figure 5. The sealed audit stream (ADR-0010): evidence of what happened, independent of the database host.

Only the durable database audit write may refuse a request; sealing and export run on a single-instance job behind an advisory lock and can fail without affecting care. Until checkpoints leave the database's host, they prove only what that host did not rewrite; on AWS they go to S3 Object Lock.

7. Real-time events

Notifications, messages and alerts are pushed over server-sent events, which work through proxies without a WebSocket upgrade. Each event has an audience: the **patient** only (their reminders), the **care team** (the patient and clinicians with an active grant, care relationship or emergency grant), **named accounts** (a message's recipients) or **administrators** (security alerts). An event never goes to everyone. Read and removed marks are stored per user, so a notification cleared on one device is cleared on all.

8. Data and clinical logic

- **Storage:** encrypted documents and queryable clinical data both live in PostgreSQL; there is no separate file store.
- **Clinical scoring is server-side:** qSOFA, SOFA, TIMI, START triage, Morse falls risk, Parkland burns fluids, Aldrete recovery, vital-sign bands and critical lab values are computed in one module and returned to the page, which only displays them. Thresholds a form needs come from a catalogue endpoint.
- **Unknown is not zero:** a score with missing inputs is returned as null with the number of criteria measured.
- **Patient visibility:** every endpoint that writes a record about a patient has a patient-scoped read.
- **Time:** reminders run on the clinic clock (Africa/Johannesburg).

9. Quality engineering

Layer	What runs	Result
Server tests	Unit and integration, against a live PostgreSQL	1,045 passing
Client tests	Vitest + Testing Library, both apps	855 staff, 280+ patient
Role journeys	Every role completes its own shift through the API with each page's real payload; a *different* user reads it back	276 steps
Browser tests	Playwright: emergency QR with a camera, live messaging on a phone, notifications, cross-role workflows	passing
Static gates in CI	35 checks, for example: no SQL concatenation, an endpoint inventory with authorisation, forms that send only what was entered, colour contrast, function size	0 failures

10. Operations

- **Backups:** pgBackRest with two repositories (local and S3), continuous WAL archiving, nightly backups at 01:47 UTC, and an automatic restore test.
- **Monitoring:** container logs go to CloudWatch; alarms on API errors, sign-in failures, account locks and host failure (with automatic recovery) e-mail the operator.
- **Cost:** one t3.small instance with S3, under a USD 60 per month budget alarm.

11. Known limits and next steps

- Phone-to-phone NFC needs a native Android app; the web supports reading NFC cards on Android Chrome only.
- One instance: fine for a pilot, while multi-instance scaling needs the sealing job's lock (already in place) and a managed database.
- Real patient data waits on POPIA authorisation and legal sign-off.

Measurements taken 20 October 2026 to 23 October 2026.